Universidad Autónoma de Baja California

Facultad de Ciencias Quimicas e Ingenieria

Ingeniería en Computación

Navarrete Pérez Andrés - 1284707

Proyecto de carrera

Taller 5



Taller 5. Prueba de concepto y planeación

Proyecto: ChiTito Smart Dispenser

Sistema IoT para el monitoreo de alimento y agua en un dispensador inteligente para mascotas.

Reporte técnico de avance y planeación.

1. Resumen ejecutivo

Se desarrolló una prueba de concepto de ChiTito Smart Dispenser para comprobar la viabilidad de una arquitectura de Internet de las Cosas que captura datos físicos, los transmite por Internet, los almacena en una base de datos y los presenta en una aplicación web. La versión implementada utiliza un ESP32, una celda de carga con convertidor HX711 y un sensor de distancia VL53L1X. La comunicación hacia un VPS de OVHcloud se realiza mediante Wi-Fi y MQTT protegido con TLS; en el servidor se emplean Mosquitto, un proceso de consumo MQTT en Laravel, MySQL y una interfaz Vue 3.

La evidencia gráfica de la interfaz, capturada el 13 de octubre de 2026 a las 11:33:32 p. m. (hora mostrada de Tijuana), presenta el dispositivo como conectado, una lectura de peso de -91 unidades crudas, una distancia de 123 mm y una estimación de alimento disponible de 71 %. La interfaz fue probada en un teléfono por el responsable del proyecto. Estas observaciones apoyan el funcionamiento del flujo, aunque todavía no sustituyen el registro experimental de diez transmisiones identificadas individualmente ni una medición instrumental de la latencia sensor–frontend.

El alcance de este taller es deliberadamente reducido: no se incluyen aún identificación RFID, servomotores, dosificación automática o control del agua. Se documentan tanto los componentes existentes como los ensayos y evidencias que faltan reunir antes de la entrega definitiva.

2. Objetivo y alcance de la prueba

Objetivo: demostrar que una magnitud adquirida de un sensor físico puede llegar desde el ESP32 hasta un backend desplegado en la nube, persistirse en MySQL y consultarse mediante una pantalla web funcional. Como variables se seleccionaron la lectura cruda

de la celda de carga, la distancia al alimento y el porcentaje estimado de contenido de la tolva.

Criterio mínimo de demostración: observar una variación real en el sensor, verificar la publicación MQTT asociada a device01, identificar el registro de recepción en MySQL y comprobar que la misma lectura, con fecha y hora, aparece en el dashboard. Para acreditar el requisito cuantitativo también deben identificarse al menos diez registros reales consecutivos y contabilizar las transmisiones correctas y fallidas.

3. Arquitectura implementada y flujo de información

La aplicación está desplegada en un VPS de OVHcloud accesible en https://15.235.63.174 mediante HTTPS. El broker Mosquitto opera separadamente y utiliza MQTT con TLS en el puerto 8883. La aplicación web se organizó en cuatro servicios Docker nuevos: Laravel, consumidor MQTT, MySQL y Nginx/Vue; la instalación original de Mosquitto se conservó. La publicación pública y el certificado HTTPS corresponden a una actualización posterior al contenido inicial del archivo ZIP, según el reporte de despliegue y la comprobación del dashboard proporcionados por el responsable.

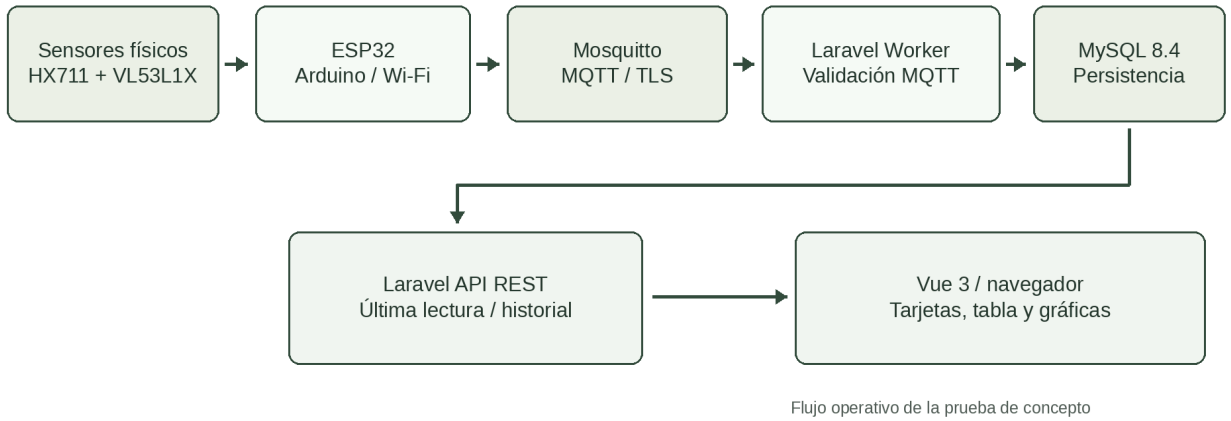


Figura 1. Flujo de la prueba de concepto: adquisición, transporte MQTT, validación, persistencia y consulta web. Elaboración propia con base en el firmware y el backend.

El ESP32 envía telemetría aproximadamente cada dos segundos al tópico chitito/device01/telemetry. Mosquitto recibe las publicaciones; el comando persistente php artisan mqtt:consume, implementado en Laravel, las valida y las almacena.

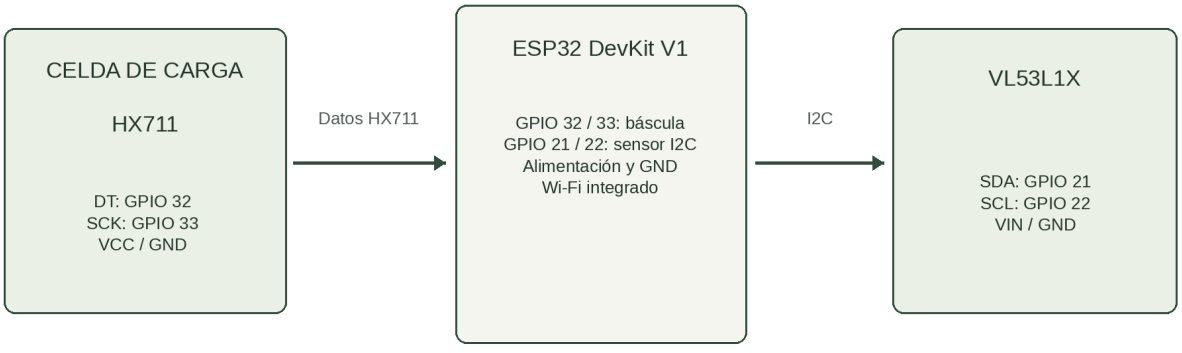
Posteriormente el frontend solicita el valor más reciente y el historial mediante la API REST autenticada. El navegador consulta la información del dispositivo cada tres segundos y actualiza el historial y las gráficas aproximadamente cada quince segundos.

4. Sistema embebido mínimo

4.1 Componentes y conexión

El programa de la versión de prueba se encuentra en main/main.ino y está desarrollado en C++ con el entorno Arduino para ESP32. Para el peso se emplea una celda de carga conectada al módulo HX711; para el nivel de alimento se emplea el sensor de distancia VL53L1X mediante I2C. Las conexiones de señales se definieron en el firmware de la siguiente manera:

Elemento	Señal	ESP32 / dato
HX711	DT (datos)	GPIO 32
HX711	SCK (reloj)	GPIO 33
VL53L1X	SDA (I2C)	GPIO 21
VL53L1X	SCL (I2C)	GPIO 22
Ambos módulos	Alimentación	3.3 V y tierra común; verificar la versión del módulo



Señales GPIO tomadas del firmware main.ino. La conexión de la celda al módulo HX711 y los niveles de alimentación se verifican según el módulo físico utilizado.

Figura 2. Diagrama de conexión lógica del prototipo. Los GPIO proceden de main.ino; el cableado de excitación/señal de la celda de carga debe seguir el módulo HX711 utilizado.

4.2 Lectura y procesamiento

El HX711 se inicializa con una espera máxima de 1 500 ms y realiza una tara de 20 muestras al arrancar. Una vez habilitado, el ESP32 consulta el módulo cada 300 ms y calcula la lectura a partir del promedio de tres muestras menos el valor de tara. Por ahora el resultado se conserva como entero crudo con signo; no representa gramos porque el sistema todavía no dispone de calibración metrológica.

El VL53L1X funciona por I2C a 400 kHz, en modo Short y con un presupuesto de medida de 50 ms. El firmware consulta su resultado cada 500 ms y lo expresa en milímetros. El porcentaje de alimento se calcula de acuerdo con dos referencias provisionales: 50 mm se interpreta como 100 % y 300 mm como 0 %. Entre ambos extremos se efectúa una interpolación lineal entera, acotada entre 0 y 100 %. Esta estimación depende de la geometría real de la tolva y deberá ajustarse en una etapa posterior.

La lectura observada de 123 mm y 71 % es coherente con esa transformación. Por ejemplo, una interpolación ideal da aproximadamente 70.8 %, que al operar con la función entera del firmware se presenta como 71 %. Esto no equivale a una calibración volumétrica de alimento.

4.3 Conectividad y formato enviado

El ESP32 se conecta en modo estación Wi-Fi, sincroniza el reloj por NTP y establece una conexión MQTT sobre TLS utilizando el certificado de autoridad configurado en su archivo privado secrets.h. El dispositivo publica su estado online/offline en chitito/device01/status. El mensaje de telemetría contiene el identificador, peso crudo, distancia y nivel, sin información personal ni credenciales.

El formato JSON que espera el backend es el siguiente; los valores son ilustrativos del contrato y no constituyen una segunda medición experimental:

```
{"device":"device01","peso_raw":1955,"distancia_mm":172,"nivel_pct":52}
```

El mensaje no contiene fecha propia; el backend asigna received_at con el reloj del servidor al guardar cada publicación. Por tanto, la marca de tiempo identifica la recepción, no exactamente el instante de adquisición. El número autoincremental id de MySQL permite individualizar cada registro.

5. Backend desplegado en la nube

El backend está implementado con Laravel, PHP y un consumidor MQTT persistente dentro de Docker. El consumidor se conecta a Mosquitto mediante TLS y cuenta de lectura, se suscribe a los tópicos de estado y telemetría, y realiza reconexiones automáticas ante fallos. El servicio de ingesta TelemetryIngestor.php interpreta el JSON, valida los campos y realiza una operación transaccional para insertar la medición y actualizar la última comunicación del dispositivo.

5.1 Validación de datos

Campo	Tipo aceptado	Regla implementada
device	Cadena	Debe coincidir con device01.
peso_raw	Entero o nulo	Rango de entero firmado de 32 bits.
distancia_mm	Entero o nulo	De 0 a 65 535 mm.
nivel_pct	Entero o nulo	De 0 a 100 %.
Mensaje completo	JSON	Tamaño máximo 2 048 bytes; se rechaza JSON inválido, tópico desconocido o campo obligatorio faltante.

Una publicación válida produce una inserción en measurements; una publicación con campos incorrectos, por ejemplo nivel_pct = 130, se rechaza mediante una excepción de validación y se registra una advertencia limitada en los logs. Al tratarse de MQTT no hay una respuesta HTTP directa al ESP32 para cada publicación; la aceptación se comprueba mediante el registro en MySQL y los registros del consumidor. Los mensajes retained de telemetría se ignoran para evitar volver a almacenar una lectura antigua como reciente.

5.2 Consulta para el frontend

La API web ofrece GET /api/telemetry/latest para recuperar la última lectura, GET /api/telemetry/history para consultar el historial paginado y GET /api/telemetry/chart para obtener datos agregados para las gráficas. GET /api/device aporta el estado del ESP32 y su última comunicación. Estas rutas requieren una sesión autenticada; las peticiones incorrectas reciben las respuestas de error convencionales de Laravel, como 401 para falta de autorización y 422 para parámetros de consulta no válidos.

En la documentación técnica entregada por Codex se reportaron 13 pruebas de Laravel/MySQL con 42 comprobaciones y ocho pruebas del frontend. También se reportaron pruebas aisladas de mensajes MQTT malformados, recepción retained y reconexión. Estas comprobaciones son de software y no equivalen a diez transmisiones físicas medidas en producción.

6. Base de datos mínima

La base implementada utiliza MySQL 8.4 y persiste sus datos mediante un volumen Docker. La migración del proyecto contiene seis tablas de aplicación y soporte: users, devices, measurements, sessions, cache y cache_locks; Laravel agrega además su tabla técnica migrations al ejecutar las migraciones, por lo que el esquema puede mostrar siete tablas en total.

La tabla central para esta práctica es measurements. Cada fila almacena el id del registro, el dispositivo asociado (device_id), peso_raw, distancia_mm, nivel_pct y received_at. El id permite distinguir dos mediciones con los mismos valores; device_id establece la relación con devices y received_at proporciona una marca de tiempo de recepción en UTC.

Tabla	Finalidad en el prototipo
measurements	Historial persistente de peso crudo, distancia, nivel y momento de recepción.
devices	Identificador único del ESP32, estado MQTT y última comunicación.
users	Cuenta de acceso para el dashboard.
sessions	Sesiones del usuario autenticado.

Tabla	Finalidad en el prototipo
cache y cache_locks	Información técnica temporal y coordinación de caché.
migrations	Control de versiones del esquema de Laravel.

6.1 Consulta de evidencias de almacenamiento

La base de datos se puede examinar sin publicar el puerto 3306, entrando al contenedor db mediante Docker. Para comprobar el requisito de diez registros reales se propone ejecutar en MySQL las siguientes consultas y conservar sus resultados como evidencia:

```
SELECT COUNT(*) AS total_mediciones FROM measurements;
SELECT id, device_id, peso_raw, distancia_mm, nivel_pct, received_at
FROM measurements ORDER BY received_at DESC, id DESC LIMIT 10;
```

El archivo ZIP proporcionado documenta que, al cierre del despliegue inicial del 12 de octubre, la base de producción no contenía mediciones porque el ESP32 estaba desconectado. La captura del 13 de octubre ya muestra lecturas en el dashboard, lo que indica una etapa posterior de funcionamiento. Sin embargo, no se adjuntó la salida actual de las consultas anteriores ni un volcado de las filas; por ello no se afirma todavía un total de diez registros verificados.

7. Frontend mínimo funcional

El frontend está desarrollado con Vue 3, Vite y Chart.js, servido por Nginx. La pantalla Vista general muestra el estado de conexión del dispositivo, la última comunicación y tres tarjetas: lectura de la báscula HX711 en unidades crudas, distancia VL53L1X en milímetros y porcentaje de alimento disponible. También ofrece gráficas de evolución de las lecturas y una vista de historial con filtros y paginación.

La interfaz solicita datos reales a Laravel; no requiere que los valores estén incrustados manualmente en la página. Maneja estados sin información, fallos de red y lecturas antiguas. El usuario confirmó que la página abre correctamente desde un teléfono. La captura proporcionada demuestra la visualización de las tres variables y su marca de tiempo.

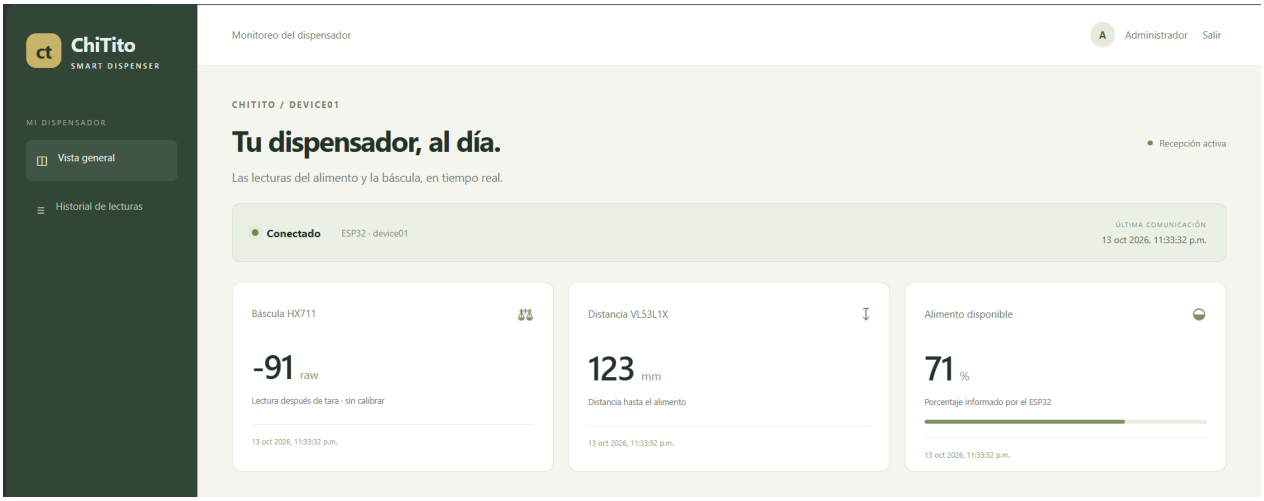


Figura 3. Captura real del dashboard ChiTito facilitada por el responsable. Lecturas mostradas el 13 de octubre de 2026, 11:33:32 p. m.: -91 raw, 123 mm y 71 %.

El acceso de producción comunicado es https://15.235.63.174, mediante HTTPS y autenticación privada. Se conserva la separación entre el frontend público, los servicios internos y el broker MQTT. La captura de la página constituye evidencia de la interfaz, pero no sustituye por sí sola el video físico ni la consulta SQL de diez filas.

8. Control de actuadores: justificación del aplazamiento

En el alcance total de ChiTito se contempla dosificar alimento y agua mediante actuadores y reconocer a las mascotas con RFID. Sin embargo, la arquitectura de monitoreo todavía debe completar la calibración de la báscula, la verificación de mediciones persistidas y las pruebas de seguridad del mecanismo. Activar servomotores o bombas sin límites físicos verificados podría provocar raciones incorrectas, atascos o funcionamiento continuo.

Por ello se propone aplazar la demostración de control remoto hasta el prototipo alfa y obtener la autorización del docente, como permite la consigna cuando existan razones técnicas justificadas. El control futuro tendrá confirmación de ejecución, límites de tiempo, protección ante fallos de red y validación de las cantidades entregadas; ninguna de estas funciones se presenta como implementada en el Taller 5.

9. Pruebas mínimas y análisis de resultados

9.1 Evidencia y estado de cada prueba

Prueba exigida	Resultado disponible	Estado de evidencia
Dato físico hasta la web	Captura con dispositivo conectado y tres lecturas; uso en celular informado por el responsable.	Observado en frontend; falta registro simultáneo serial y SQL.
Diez transmisiones consecutivas	El firmware publica cada 2 s y MySQL crea una fila por mensaje aceptado.	Pendiente: IDs y marcas de tiempo de diez registros reales.
Éxitos y fallos	Existen validaciones y pruebas MQTT aisladas reportadas por Codex.	Pendiente: conteo real de éxitos/fallos en una corrida física.
Latencia sensor–frontend	Se conocen periodos de muestreo/publicación y actualización de interfaz.	Estimación de diseño; falta cronometraje instrumental.
Dato incorrecto	El consumidor rechaza valores fuera de rango y JSON inválido; pruebas aisladas reportadas.	Comprobado en entorno de prueba; falta evidencia de ejecución documentada.
Pérdida de conexión	El consumidor incorpora reconexión y el frontend reporta desconexión/antigüedad.	Pruebas aisladas reportadas; falta ensayo físico de desconexión Wi-Fi.

9.2 Protocolo para diez transmisiones consecutivas

Conectar el ESP32, abrir el monitor serie a 115 200 baudios e identificar la salida MQTT de cada publicación. Mantener el dispositivo enviando durante al menos 20 segundos, sin interrumpir la red. Anotar el instante de inicio y consultar la base para localizar los diez registros de ese intervalo, ordenados por received_at e id. Contabilizar como transmisión exitosa aquella que tenga publicación observable y fila correspondiente; marcar como

fallida la que no llegue a MySQL durante el periodo de prueba. Repetir con cambios controlados de peso o nivel para demostrar adquisición física y no solamente transmisión repetida de valores.

Las cantidades n_correctas y n_fallidas deberán obtenerse de esa corrida; la tasa de éxito se calcula como 100 × n_correctas / 10. En el material disponible no existe una bitácora que permita asignar legítimamente esos resultados a diez intentos individuales, así que no se presenta una tasa numérica no medida.

9.3 Tiempo aproximado sensor–frontend

El firmware consulta el HX711 cada 300 ms y el VL53L1X cada 500 ms; publica cada 2 000 ms y la pantalla solicita el dato más reciente cada 3 000 ms. Por diseño, una lectura puede esperar hasta aproximadamente 0.5 s por el siguiente muestreo, 2 s para la próxima publicación y 3 s para la siguiente consulta del navegador, además de los tiempos variables de red, broker, backend, base de datos y renderizado. La suma de estas ventanas ofrece una cota orientativa de alrededor de 5.5 s más las latencias de procesamiento, no una medición experimental ni una garantía de tiempo máximo.

Para obtener un valor medido debe sincronizarse el reloj de los equipos, registrar el instante de lectura en el ESP32, el valor received_at en MySQL y la aparición de la lectura en el navegador. Se sugiere realizar diez repeticiones y reportar promedio, mínimo y máximo. El reloj del backend guarda UTC y la interfaz convierte la hora a America/Tijuana.

9.4 Ensayos de datos inválidos y conexión

El ensayo negativo más sencillo consiste en publicar desde un cliente MQTT de prueba, con credenciales de laboratorio y sin alterar el firmware, un mensaje cuyo nivel_pct sea 130. El servicio deberá rechazarlo sin insertar una fila en measurements y registrar un error controlado. Otro ensayo consiste en apagar temporalmente el ESP32 o retirar Wi-Fi, verificar que el frontend indique desconexión o datos antiguos y confirmar que la comunicación se recupere al restablecer Internet. La prueba aislada de reconexión del consumidor ya figura en la entrega de Codex, pero debe repetirse con el equipo físico para cerrar este requisito.

9.5 Problemas identificados y correcciones

Problema	Medida aplicada o recomendada
Lectura HX711 sin calibración	Se muestran unidades raw sin afirmar gramos; calibrar después con masas conocidas.
Fallas de conectividad o MQTT	Se implementaron reconexiones, Last Will y estados de antigüedad; medir pérdidas en pruebas reales.
Mensajes inválidos o retained	Validación en Laravel; rechazo de telemetría retained para no duplicar registros viejos.
Sensor detenido con valor previo	El firmware conserva el último valor si falla una lectura; agregar estado de salud por sensor.
Fallo NTP al inicio	El firmware podría continuar con MQTT incompleto; diseñar reintento NTP sin bloquear el sistema.
Datos crecientes en MySQL	Índice por dispositivo y fecha, consultas paginadas; preparar política de respaldo y retención.

10. Evidencias de implementación

Se cuenta con código fuente inicial, documentación técnica y captura funcional. El paquete del proyecto contiene main/main.ino, código de Laravel y Vue, Docker Compose, configuración de Nginx, archivos de pruebas y un README básico. La bitácora de Codex describe cuatro servicios web desplegados y las pruebas de software realizadas. El responsable informó que la versión pública con HTTPS está accesible desde computadora y teléfono.

Evidencia solicitada	Disponibilidad y fuente
Código fuente inicial	Disponible en ChiTito.zip: main/main.ino, backend/, frontend/, infrastructure/.
Diagrama de conexión y flujo	Incluidos como Figuras 1 y 2 de este reporte.
Video breve de funcionamiento	Pendiente de grabar con el hardware, monitor serie, dashboard y consulta SQL.

Evidencia solicitada	Disponibilidad y fuente
Backend desplegado	Reportado por Codex en el VPS OVHcloud; falta anexar captura actual de docker compose ps para evidencia visual.
Estructura inicial de base de datos	Migración Laravel y tablas descritas en el apartado 6; falta evidencia SQL actual de diez filas.
Frontend con datos reales	Captura proporcionada: Figura 3, dispositivo online y mediciones visibles.
Repositorio Git y README	README y .gitignore presentes; no se proporcionó URL ni evidencia de publicación del repositorio remoto.
Cronograma y presupuesto	Incluidos en los apartados 11 y 12.
Matriz de riesgos	Incluida en el apartado 13.

10.1 Guion de video de demostración

Se propone un video de 60 a 90 segundos que muestre, en secuencia y sin sustituir la ejecución real por capturas estáticas: (1) placa ESP32 y ambos sensores conectados; (2) interacción física con la celda de carga o cambio de distancia frente al VL53L1X; (3) monitor serie con la publicación MQTT y su JSON; (4) dashboard mostrando la lectura actualizada y su fecha; (5) consulta SELECT de MySQL con diez registros recientes; (6) breve corte de conexión y recuperación, si el tiempo lo permite. Debe registrarse la misma sesión de prueba para facilitar la correlación entre evidencias.

11. Cronograma de las semanas restantes

Como no se proporcionó el calendario exacto del curso, se propone un plan de seis semanas, del 19 de octubre al 29 de noviembre de 2026, para organizar la transición de la prueba de concepto al prototipo alfa. Las fechas son una propuesta de trabajo y deberán ajustarse a las sesiones reales del taller.

Semana	Actividades	Entregable verificable
1 · 19–25 oct.	Capturar diez transmisiones, latencia, fallos y video; respaldo de código y resultados SQL.	Bitácora experimental, video y evidencia de base.

Semana	Actividades	Entregable verificable
2 · 26 oct.–1 nov.	Calibrar la celda de carga y revisar geometría del depósito; validar rangos físicos.	Conversión a gramos probada y porcentaje ajustado.
3 · 2–8 nov.	Integrar lector RFID y diseñar estados seguros de reconocimiento de mascotas.	Identificación local demostrada y pruebas de errores.
4 · 9–15 nov.	Montar mecanismo de alimento y servomotores; validar límites y confirmaciones.	Prototipo alfa de dispensación con paro seguro.
5 · 16–22 nov.	Integrar nivel y calidad indicativa de agua, bomba y mediciones auxiliares.	Subsistema de agua probado por separado.
6 · 23–29 nov.	Integrar subsistemas, corregir fallos, mejorar frontend, documentar y presentar.	Demostración integrada, pruebas y reporte final.

Se prioriza terminar y documentar el flujo actual antes de activar actuadores. Cualquier ajuste de alcance deberá preservarse en la bitácora, incluyendo decisiones de seguridad y la autorización del docente para posponer controles remotos.

12. Presupuesto preliminar

Los siguientes montos son estimaciones de planeación en pesos mexicanos, no facturas ni comprobantes de compra. Se distinguen los elementos de la prueba actual, una posible ampliación al prototipo alfa y el servicio mensual de nube. El hardware existente no implica que todos los artículos se hayan adquirido a esos precios. Como referencia de mercado, módulos VL53L1X se ofrecen en México aproximadamente en el rango de 200 a 250 MXN según proveedor y disponibilidad.

Etapa	Componente	Cant.	Unitario	Subtotal
Prueba actual	ESP32 DevKit V1	1	$200	$200

Etapa	Componente	Cant.	Unitario	Subtotal
Prueba actual	Módulo HX711	1	$70	$70
Prueba actual	Celda de carga 5 kg	1	$110	$110
Prueba actual	Módulo VL53L1X	1	$240	$240
Prueba actual	Protoboard y cables	1	$120	$120
Prueba actual	Cable y fuente USB	1	$100	$100
Prueba actual	Contenedor sencillo de prueba	1	$90	$90
Etapa futura	Lector RFID RC522	1	$90	$90
Etapa futura	Servo MG90S	2	$130	$260
Etapa futura	Sensor de nivel de agua A02YYUW	1	$180	$180
Etapa futura	Sensor TDS SEN0244	1	$400	$400
Etapa futura	Sensor de turbidez SEN0189	1	$300	$300
Etapa futura	DS18B20 y accesorios	1	$90	$90
Etapa futura	Bomba de agua de 5 V	1	$150	$150
Etapa futura	MOSFET, diodo y protecciones	1	$80	$80
Etapa futura	Fuente de alimentación de potencia	1	$170	$170

Etapa	Componente	Cant.	Unitario	Subtotal
Etapa futura	Estructura, depósito, tolva y conductos	1	$700	$700
Etapa futura	Conectores y protecciones mecánicas	1	$250	$250

Concepto	Monto estimado
Materiales de la prueba de concepto	$930 MXN
Materiales para funciones futuras	$2,670 MXN
Subtotal de hardware	$3,600 MXN
Reserva para imprevistos (15 % del hardware)	$540 MXN
VPS OVHcloud: provisión de 2 meses (250 MXN/mes)	$500 MXN
Total preliminar del proyecto	$4,640 MXN

El costo del VPS es una provisión de presupuesto; no se proporcionó la factura ni el plan contratado, por lo que debe sustituirse por el importe real. Laravel, Vue, MySQL, Mosquitto, Docker y Let’s Encrypt no requieren pago por licencias en esta implementación. Tampoco se incluye mano de obra, envíos ni consumo eléctrico, y el proyecto puede cambiar conforme avancen las pruebas de integración.

13. Matriz de riesgos y acciones de respuesta

Riesgo	Prob. / Impacto	Prevención y contingencia
Pérdida de Wi-Fi o Internet	Media / Alta	Reconexión y estado offline; registrar cortes y reintentar al restablecer la red.
Certificado MQTT o HTTPS vencido	Media / Alta	Monitorear vigencias y renovación; probar verificación TLS y documentar recuperación.
Deriva o calibración incorrecta de HX711	Alta / Media	Tara controlada y masas patrón; impedir uso de gramos hasta verificar calibración.
Geometría de tolva distinta a la asumida	Media / Media	Medir niveles lleno/vacío con alimento real; ajustar fórmula y marcar incertidumbre.

Riesgo	Prob. / Impacto	Prevención y contingencia
Mensajes perdidos por QoS 0	Media / Media	Bitácora de publicaciones/filas; valorar QoS 1 e identificadores secuenciales en alfa.
Fallas de bomba o servo / bloqueo mecánico	Media / Alta	Etapas posteriores con límites de tiempo, protección de alimentación y parada segura.
Crecimiento de registros o falla de disco	Media / Alta	Índices y respaldo automático; monitorear almacenamiento y ensayar restauración aislada.
Acceso no autorizado al dashboard	Media / Alta	HTTPS, sesiones privadas, contraseñas robustas, mínimos privilegios y actualizaciones.
Atraso de integración de sensores y actuadores	Alta / Media	Entregables semanales priorizados; congelar alcance si falla una dependencia crítica.

14. Conclusiones

La prueba de concepto permite comprobar la viabilidad técnica de transportar información de sensores reales desde un ESP32 hacia un servidor en la nube y presentarla en una interfaz web. La separación entre adquisición, mensajería MQTT, procesamiento Laravel, persistencia MySQL y visualización Vue 3 facilita la ampliación del proyecto sin reemplazar la infraestructura existente. La interfaz pública mostrada y la documentación del despliegue evidencian un avance superior al mínimo exigido para la pantalla y el backend.

El principal resultado todavía pendiente de documentar con trazabilidad completa es la demostración experimental de diez transmisiones consecutivas, con sus identificadores SQL, porcentaje de éxito y tiempos sensor–frontend. De igual forma, falta grabar el video que vincule el hardware físico, el monitor serial, el registro en base de datos y la pantalla. Estos faltantes se identificaron expresamente para que la entrega no confunda pruebas automatizadas o una captura web con un ensayo físico completo.

Para las siguientes semanas se propone cerrar primero la trazabilidad y calibración, y avanzar después hacia identificación de mascotas, mecanismos de alimento y subsistema de agua. Esta secuencia reduce riesgos técnicos y permite evaluar cada componente de forma independiente antes de integrarlo en el dispensador definitivo.

15. Referencias

[1] ChiTito Smart Dispenser. “main/main.ino” y “docs/FIRMWARE-MQTT.md”, archivos del proyecto suministrados en ChiTito.zip, octubre de 2026.

[2] ChiTito Smart Dispenser. “docs/ENTREGA.md”, “docs/API.md”, código de Laravel, migraciones y frontend Vue; archivos del proyecto suministrados, octubre de 2026.

[3] Evidencia fotográfica del responsable del proyecto. Captura de pantalla del dashboard ChiTito, 13 de octubre de 2026.

[4] Avia Semiconductor. “HX711: 24-Bit Analog-to-Digital Converter for Weigh Scales”, hoja de datos. https://cdn.sparkfun.com/assets/learn_tutorials/5/4/6/hx711F_EN.pdf

[5] STMicroelectronics. “VL53L1X: Time-of-Flight Ranging Sensor”, documentación técnica. https://www.st.com/content/st_com/en/products/imaging-and-photonics-solutions/time-of-flight-sensors/vl53l1x.html

[6] Eclipse Mosquitto. “mosquitto-tls: Configure SSL/TLS support for Mosquitto”. https://mosquitto.org/man/mosquitto-tls-7.html

[7] Laravel. “Laravel 13.x — Database: Query Builder”. https://laravel.com/framework/docs/13.x/queries

[8] Docker. “Define and manage volumes in Docker Compose”. https://docs.docker.com/reference/compose-file/volumes/

[9] Lionchip México. “Sensor de distancia 400 cm VL53L1X”, referencia comercial de precio; consulta de octubre de 2026. https://www.lionchipmexico.com/product-page/modulo-vl53l1x-sensor-de-distancia-laser-400-cm